perplexity

Step 5: Tools, automation, and best practices

Automation makes regression testing practical at scale: instead of manually repeating the same checks, teams run stable test scripts automatically in a CI/CD pipeline after code changes. The goal is fast, trustworthy feedback—not simply automating every possible test.[1] [2]

Common tools

Testing layer	Tools	What they check
Unit tests	JUnit (Java), Pytest (Python), Jest (JavaScript)	Individual functions, classes, or components
API/integration tests	REST Assured, Postman/Newman, Keploy	API responses, service communication, data rules
Web UI tests	Playwright, Cypress, Selenium	Browser-based user journeys such as login and checkout
Mobile tests	Appium	Mobile app functionality across devices
CI/CD orchestration	GitHub Actions, GitLab CI/CD, Jenkins	When and how tests run automatically [3] [4] [5]

For a beginner building web-app regression tests, **Playwright** or **Cypress** are common modern choices; **Selenium** remains widely used, especially for broad browser and language support. For Python, **Pytest** is a strong starting point for unit and API-level regression tests.[3] [5] [6]

How CI/CD automation works

A typical pipeline follows this flow:

1. A developer opens a pull request.
2. The CI system builds the application and starts a test environment.
3. It runs fast smoke, unit, and targeted regression tests.
4. If tests pass, the change can be merged; if they fail, the developer receives logs and failure details.
5. A broader regression suite may run after merge, nightly, or before release.[5] [7] [1]

For example, in GitHub Actions, a workflow can trigger on every pull request, run `pytest` or Playwright tests, annotate the pull request with pass/fail results, and prevent merging when critical tests fail. GitLab CI/CD and Jenkins offer similar automation through pipeline jobs and plugins.[4] [1]

What to automate first

Start with tests that are repeatable, stable, high-value, and fast. Critical user journeys—such as signup, login, search, checkout, and payment—are usually better early automation candidates than rarely used or highly visual features. [2] [8]

Good first candidates:

- Login and logout
- Core navigation
- Search and filtering
- Add-to-cart and checkout
- Payment success and failure paths
- Important API responses and error handling [8] [2]

Avoid automating everything immediately. Exploratory testing, subjective visual design, and frequently changing features often benefit more from human judgment. [2] [8]

Avoiding flaky tests

A **flaky test** passes sometimes and fails other times without a real product defect. Flaky tests reduce trust in the regression suite because teams start ignoring failures. [7] [9]

Common causes and fixes:

Cause	Fix
Slow page loading or timing issues	Use automatic waits rather than fixed sleep delays
Unstable selectors	Use stable attributes such as `data-testid` or ARIA roles instead of CSS classes
Shared test data	Isolate test data and reset state between tests
External service failures	Mock unstable dependencies or use controlled test environments
Test depends on another test	Make each test independent and able to run alone [2] [9] [8]

If a test remains flaky, teams often **quarantine** it: move it to a separate non-blocking job, assign an owner, and fix or retire it quickly rather than letting it block every deployment. [9] [7]

Best practices

- **Use a testing pyramid:** many fast unit tests, fewer integration/API tests, and fewer end-to-end UI tests.
- **Keep tests independent:** one failing test should not cause unrelated failures.
- **Use stable selectors:** prefer `data-testid` or accessible roles over visual styling classes.
- **Run tests in parallel:** this reduces execution time, especially for full regression suites.
- **Separate smoke from full regression:** run smoke tests on every change and larger suites nightly or before release.

- **Maintain the suite:** add tests for new features and bug fixes, update outdated expectations, and remove tests for removed features.
- **Track suite health:** monitor pass rate, execution time, and flakiness trends. [10] [5] [8] [2]

Beginner-friendly setup

A simple first stack could be:

- **Application:** A small web app
- **Language:** Python or JavaScript
- **Unit/API tests:** Pytest or Jest
- **Browser tests:** Playwright or Cypress
- **CI:** GitHub Actions
- **First suite:** 5–10 critical smoke and regression tests

For example, a beginner could automate: “user can log in,” “user can search for a product,” “user can add an item to cart,” and “cart total updates correctly.” Once these are stable, expand gradually to checkout, payments, and account settings. [5] [2]

Step 6 preview

Next, I can explain **Step 6: A complete beginner project**, where you build a small regression suite for a demo web app:

- Choose a demo application
- Write your first test cases
- Automate them with Playwright or Pytest
- Add them to GitHub Actions
- Interpret pass/fail results and fix a failure

Would you like to continue with **Step 6**?

⁂

1. https://www.virtuosoqa.com/post/ci-cd-regression-testing
2. https://keploy.io/blog/community/automated-regression-testing
3. https://www.ranorex.com/blog/automated-regression-testing-tools/
4. https://marker.io/blog/regression-testing-tools
5. https://www.devopssupport.in/blog/the-complete-guide-to-automated-testing-in-ci-cd-pipelines/
6. https://www.webbrowserbot.com/regression-testing/
7. https://www.harness.io/blog/regression-testing-in-ci-cd-deliver-faster-without-the-fear
8. https://qapractices.com/documentation/regression-testing/
9. https://www.radview.com/blog/understanding-regression-testing-software/

10. https://www.kualitatem.com/blog/automation-testing/a-guide-to-regression-testing-types-and-best-practices/
11. https://www.testingxperts.com/blog/regression-test-suite-optimization/
12. https://www.testrail.com/blog/regression-testing/
13. https://www.virtuosoqa.com/post/regression-testing-best-practices
14. https://www.flagsmith.com/blog/automated-regression-testing
15. https://katalon.com/resources-center/blog/automated-regression-testing-guide